

Cryptography

Difference between symmetric and asymmetric encryption:

Symmetric Key Encryption	Asymmetric Key Encryption
It only requires a single key for both encryption and decryption.	It requires two keys, a public key and a private key, one to encrypt and the other to decrypt.
The size of cipher text is the same or smaller than the original plain text.	The size of cipher text is the same or larger than the original plain text.
The encryption process is very fast.	The encryption process is slow.
It is used when a large amount of data needs to be transferred.	It is used to transfer small amount of data.
It only provides confidentiality.	It provides confidentiality, authenticity, and non-repudiation.
The length of key used is 128 or 256 bits	The length of key used is 2048 or higher
In symmetric key encryption, resource utilization is low compared to asymmetric key encryption.	In asymmetric key encryption, resource utilization is high.
It is efficient as it is used for handling large amount of data.	It is comparatively less efficient as it can handle a small amount of data.
Security is lower as only one key is used for both encryption and decryption purposes.	Security is higher as two keys are used, one for encryption and the other for decryption.
Examples: 3DES, AES, DES and RC4	Examples: Diffie-Hellman, ECC, El Gamal, DSA and RSA

Hashing

Hashing is a method that takes any type of data, such as a word, a file, or even a full message, and converts it into a short, fixed-length string of letters and numbers. This result is called a hash code.

Hashing Algorithms:

1. **Message digest 5 (MD5):** MD5 was once a very popular hashing algorithm due to its speed and simplicity. It creates a 128-bit hash and is commonly used for checking file integrity or verifying that files hadn't changed during a transfer. However, over time, serious flaws were found, especially its vulnerability to collisions, which means two different data points can end up producing the same hash.
2. **Secure hash algorithm 1 (SHA1):** SHA-1 was developed to be stronger than MD5. It produces a longer, 160-bit hash and was widely used for authenticating passwords, file verification, and digital signatures. Over time, like MD5, SHA-1 was eventually found to be weak. Google and the CWI Institute in Amsterdam publicly demonstrated a collision attack against SHA-1, known as the "SHattered" attack, where two different PDF files produced the same SHA-1 hash, which proves that this algorithm is also no longer collision-resistant.
3. **Secure hash algorithm 2:** SHA-2 is a group of hashing algorithms, with SHA-256 and SHA-512 being the most common. These algorithms are much stronger and widely used in digital

signatures and cryptocurrency. While it's slower than MD5, it's highly resistant to attacks and is considered a reliable choice for almost all cybersecurity needs.

4. **Secure hash algorithm 256:** The SHA-256 algorithm belongs to the family of the SHA 2 algorithms. The main motive behind publishing this algorithm was to create a successor for the SHA one family and it was developed by the NSA as well as the NIST organization because the SHA-1 family was losing its strength because of brute force attacks. Here are some of the most important characteristics about the SHA-256 that makes it stand out from the rest:

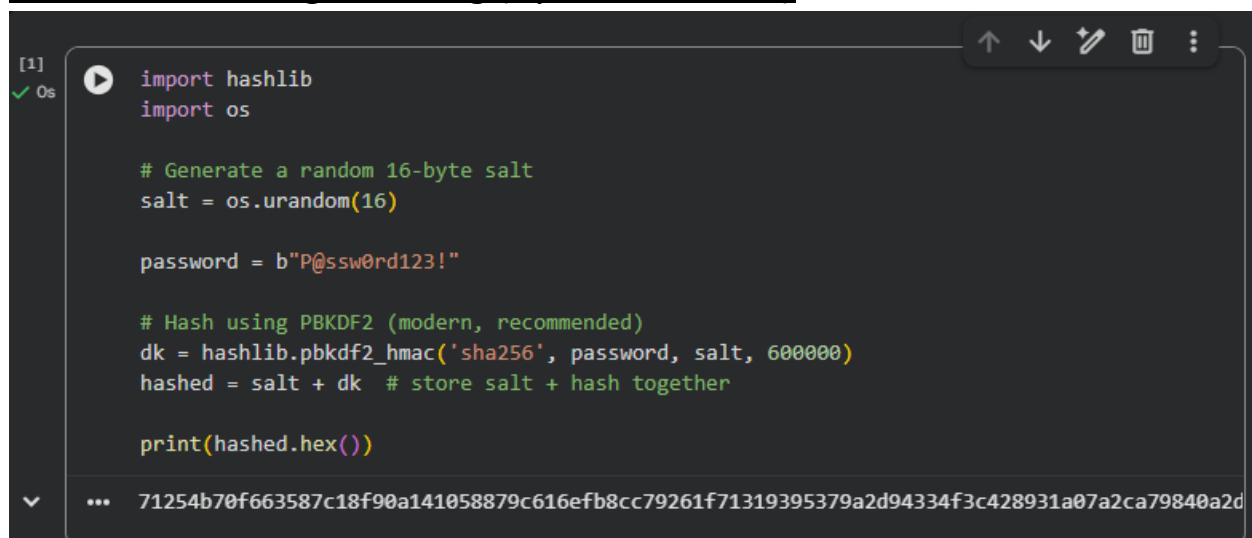
- Digest Length
- Irreversible Quality

Why MD5 is insecure

it is secure because of the following reasons:

- MD5 generates the same hash function for different inputs (**hash collision**).
- MD5 provides poor security over SHA1, SHA256 and other modern cryptographic algorithms.
- MD5 has been considered an insecure algorithm. So now we are using SHA256 instead of MD5.
- MD5 is neither a symmetric nor asymmetric algorithm.

Password Hashing & Salting (Python Practice)

A screenshot of a Python code editor with a dark theme. The code imports hashlib and os, generates a 16-byte salt using os.urandom(16), and hashes the password 'P@ssw0rd123!' using hashlib.pbkdf2_hmac with 'sha256' and 600,000 iterations. The output is a long hexadecimal string.

```
[1]
✓ Os
import hashlib
import os

# Generate a random 16-byte salt
salt = os.urandom(16)

password = b"P@ssw0rd123!"

# Hash using PBKDF2 (modern, recommended)
dk = hashlib.pbkdf2_hmac('sha256', password, salt, 600000)
hashed = salt + dk # store salt + hash together

print(hashed.hex())

... 71254b70f663587c18f90a141058879c616efb8cc79261f71319395379a2d94334f3c428931a07a2ca79840a2d84b531
```

Import the hashlib and os and a string (hash) of 16bit will be created for your password using sha256 output will be:

```
71254b70f663587c18f90a141058879c616efb8cc79261f71319395379a2d94334f3c428931a07a2ca79840a2d84b531
```

For verification use this code

```
def verify_password(stored, password_attempt):
    salt = stored[:16]
    original_hash = stored[16:]
    new_hash = hashlib.pbkdf2_hmac('sha256', password_attempt.encode(), salt, 600000)
```

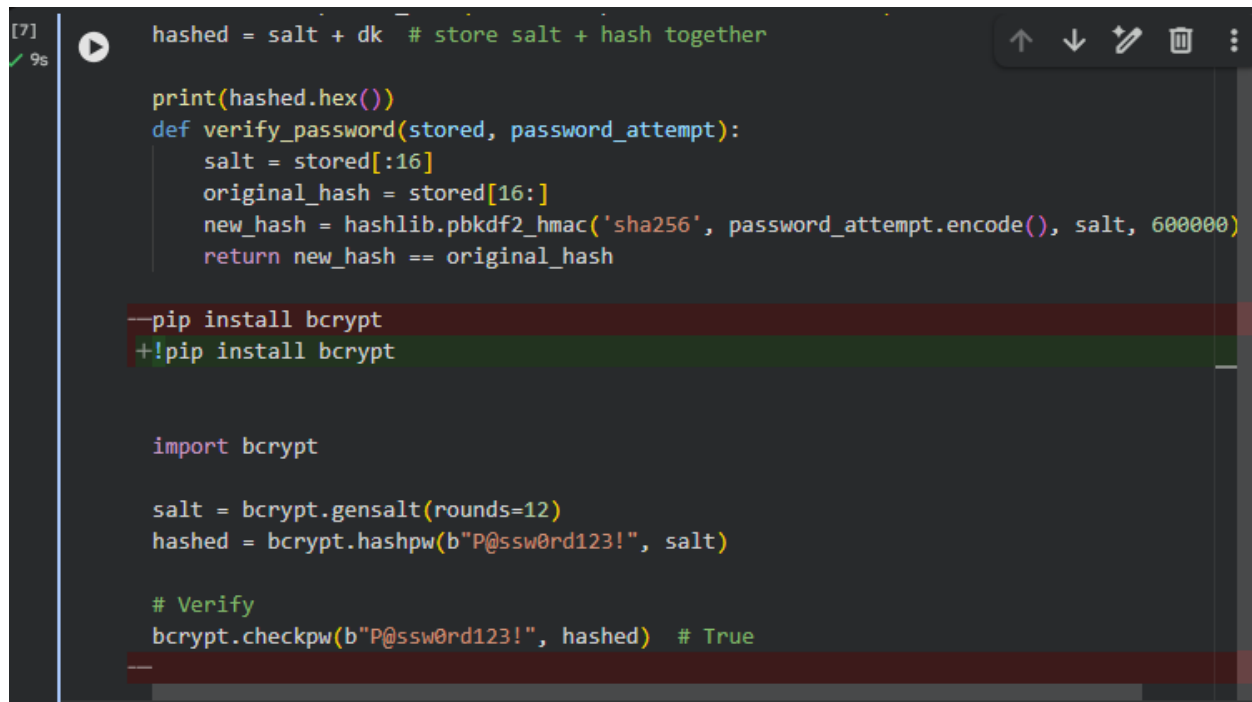
```

    return new_hash == original_hash
!pip install bcrypt
import bcrypt

salt = bcrypt.gensalt(rounds=12)
hashed = bcrypt.hashpw(b"P@ssw0rd123!", salt)

# Verify
bcrypt.checkpw(b"P@ssw0rd123!", hashed)

```



```

[7] hashed = salt + dk # store salt + hash together
✓ 9s

print(hashed.hex())
def verify_password(stored, password_attempt):
    salt = stored[:16]
    original_hash = stored[16:]
    new_hash = hashlib.pbkdf2_hmac('sha256', password_attempt.encode(), salt, 600000)
    return new_hash == original_hash

--pip install bcrypt
+!pip install bcrypt

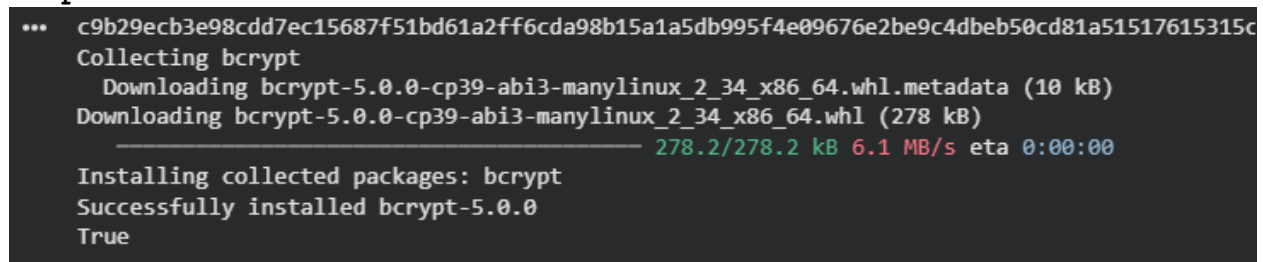
import bcrypt

salt = bcrypt.gensalt(rounds=12)
hashed = bcrypt.hashpw(b"P@ssw0rd123!", salt)

# Verify
bcrypt.checkpw(b"P@ssw0rd123!", hashed) # True

```

Output will be:



```

... c9b29ecb3e98cdd7ec15687f51bd61a2ff6cda98b15a1a5db995f4e09676e2be9c4dbeb50cd81a51517615315c
Collecting bcrypt
  Downloading bcrypt-5.0.0-cp39-abi3-manylinux_2_34_x86_64.whl.metadata (10 kB)
  Downloading bcrypt-5.0.0-cp39-abi3-manylinux_2_34_x86_64.whl (278 kB)
    278.2/278.2 kB 6.1 MB/s eta 0:00:00
Installing collected packages: bcrypt
Successfully installed bcrypt-5.0.0
True

```

Password Salting and its importance:

Password salting is a security technique used to safeguard passwords stored in databases. It involves adding a unique, random string of characters, known as a "salt," to each password before it is hashed (converted into a fixed-length string by a hash function).

This salt is then stored along with the hashed password. When the password needs to be verified, the salt is retrieved and combined with the entered password, and the hash function is run again to see if the results match the stored hash.

importance:

- **Unique Hashes:** Salting creates unique hashes for each password, preventing attackers from using precomputed hash dictionaries to crack multiple accounts simultaneously.
- **Counteracts Rainbow Table Attacks:** Rainbow tables, which contain precomputed hashes of common passwords, become ineffective because each password hash now requires a unique salt, making these attacks more resource-intensive and less feasible.
- **Complicates Brute Force Attacks:** Attackers must guess the salt in addition to the password, significantly increasing the difficulty of brute force attacks.
- **Protects Against Hash Collisions:** By adding a salt, you reduce the risk of two passwords generating the same hash (collision), enhancing overall database security.

Demonstration Of Password Cracking Concepts Using Hashcat Or John The Ripper On Sample hashes

1. Generate the sample hashes in python:

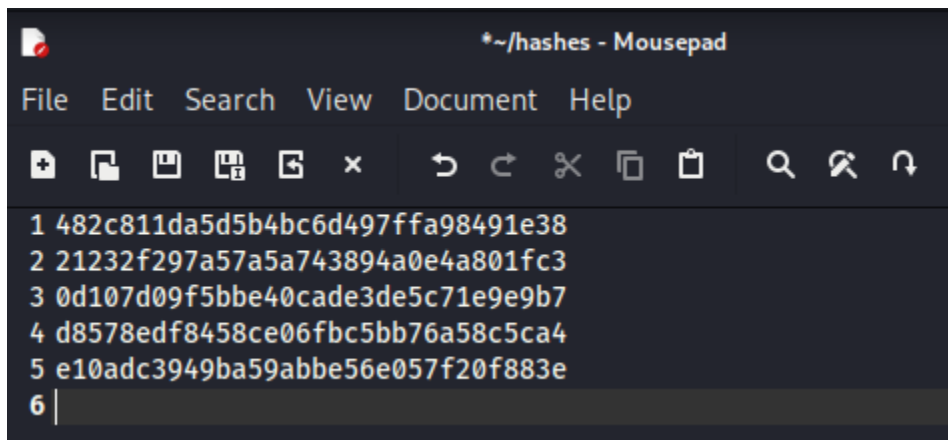


```
[8] import hashlib

passwords = ["password123", "admin", "letmein", "qwerty", "123456"]
for p in passwords:
    h = hashlib.md5(p.encode()).hexdigest()
    print(f"{h}")

... 482c811da5d5b4bc6d497ffa98491e38
    21232f297a57a5a743894a0e4a801fc3
    0d107d09f5bbe40cade3de5c71e9e9b7
    d8578edf8458ce06fbc5bb76a58c5ca4
    e10adc3949ba59abbe56e057f20f883e
```

2. Save the file in kali so can be used later.



```
*~/hashes - Mousepad
File Edit Search View Document Help

1 482c811da5d5b4bc6d497ffa98491e38
2 21232f297a57a5a743894a0e4a801fc3
3 0d107d09f5bbe40cade3de5c71e9e9b7
4 d8578edf8458ce06fbc5bb76a58c5ca4
5 e10adc3949ba59abbe56e057f20f883e
6 |
```

3. Install rockyou.txt.gz if not installed, otherwise extract it from the given command if its pre installed.

```
(root@kali123)-[/home/kali123]
# ls -lh /usr/share/wordlists/
total 51M
lrwxrwxrwx 1 root root 26 Jun 11 12:44 amass -> /usr/share/amass/wordlists
lrwxrwxrwx 1 root root 25 Jun 11 12:44 dirb -> /usr/share/dirb/wordlists
lrwxrwxrwx 1 root root 30 Jun 11 12:44 dirbuster -> /usr/share/dirbuster/wordlists
lrwxrwxrwx 1 root root 35 Jun 11 12:44 dnsmap.txt -> /usr/share/dnsmap/wordlist_TLAs.txt
lrwxrwxrwx 1 root root 41 Jun 11 12:44 fasttrack.txt -> /usr/share/set/src/fasttrack/wordlist.txt
lrwxrwxrwx 1 root root 45 Jun 11 12:44 fern-wifi -> /usr/share/fern-wifi-cracker/extras/wordlists
lrwxrwxrwx 1 root root 28 Jun 11 12:44 john.lst -> /usr/share/john/password.lst
lrwxrwxrwx 1 root root 27 Jun 11 12:44 legion -> /usr/share/legion/wordlists
lrwxrwxrwx 1 root root 46 Jun 11 12:44 metasploit -> /usr/share/metasploit-framework/data/wordlists
lrwxrwxrwx 1 root root 41 Jun 11 12:44 nmap.lst -> /usr/share/nmap/nselib/data/passwords.lst
-rw-r--r-- 1 root root 51M May 12 2023 rockyou.txt.gz
lrwxrwxrwx 1 root root 39 Jun 11 12:44 sqlmap.txt -> /usr/share/sqlmap/data/txt/wordlist.txt
lrwxrwxrwx 1 root root 25 Jun 11 12:44 wfuzz -> /usr/share/wfuzz/wordlist
lrwxrwxrwx 1 root root 37 Jun 11 12:44 wifite.txt -> /usr/share/dict/wordlist-probable.txt
(root@kali123)-[/home/kali123]
```

4. Run this command to unzip it

```
(root@kali123)-[/home/kali123]
# gunzip /usr/share/wordlists/rockyou.txt.gz
```

- 5.start the attack using john

```
(root@kali123)-[/home/kali123]
# john --format=raw-md5 --wordlist=/usr/share/wordlists/rockyou.txt hashes
Using default input encoding: UTF-8
Loaded 5 password hashes with no different salts (Raw-MD5 [MD5 256/256 AVX2 8x3])
Warning: no OpenMP support for this hash type, consider --fork=2
Press 'q' or Ctrl-C to abort, almost any other key for status
123456 (??)
qwerty (??)
letmein (??)
password123 (??)
admin (??)
5g 0:00:00:00 DONE (2026-06-16 14:15) 250.0g/s 998400p/s 998400c/s 1152KC/s 150588..jone1
Use the "--show --format=Raw-MD5" options to display all of the cracked passwords reliably
Session completed.
```

5. Cracked successfully, results are given

```
(root@kali123)-[/home/kali123]
# john --show --format=Raw-MD5 hashes
?:password123
?:admin
?:letmein
?:qwerty
?:123456

5 password hashes cracked, 0 left
```

SSL/TLS security:

SSL/TLS certificates are digital files that verify a website's identity and enable encrypted communication over the internet. They work using Public Key Infrastructure (PKI) to provide security through authentication, encryption and data integrity.

SSL (Secure Socket Layer (SSL) Certificate)

An SSL certificate is a digital certificate that can be used for authentication of a website and it creates a secure connection between the client and the web server. When installed, it enables the website to move from HTTP to HTTPS.

TLS (Transport Layer Security)

Transport Layer Security (TLS) are designed to provide security at the transport layer. TLS was derived from a security protocol called Secure Socket Layer (SSL). TLS ensures that no third party may tamper with any message. It protects against:

- **Eavesdropping** (someone spying on your connection)
- **Data tampering** (changing data in transit)
- **Impersonation** (fake websites)

Working

In the process of certificate-based authentication, when a user requests access to a protected resource, the server responds by presenting its certificate to the user's browser. The browser then verifies the authenticity of the server's public certificate. Subsequently, an authentication request is sent from the server, prompting the user to authenticate themselves. Concurrently, while the user undergoes authentication, their browser provides the server with the user's certificate for validation. Upon successful validation of the user's identity by the server, access to the network or protected resource is granted.

password security best practices guide for CoreTech Innovation

- best practice is to use a hybrid way for encryption like using symmetric technique to encrypt the whole data and use the asymmetric technique for the encryption of key generated by symmetric method.
- Use sha256
- Use salts as additional layer of security
- Last but not the least one don't rely completely on these... always keep a backup plan.... As every algorithm has its own vulnerabilities.... If not detected yet but may be in future they can be.